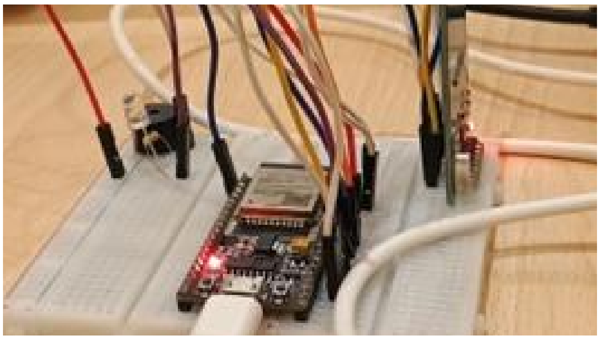
COMPUTATIONAL HARDWARE(IOT)

SCHEMATIC CAPTURE:

The project involves an IoT-based RFID access control system using an ESP32, an RC522 RFID reader, a buzzer, an LED, an LCD, a Servo motor, and a DS1302 RTC module. The schematic captures the connection details as follows:

- **RFID RC522 to ESP32:**
  - SDA → GPIO5
  - SCK → GPIO18
  - MOSI → GPIO23
  - MISO → GPIO19
  - RST → GPIO4
  - VCC → 3.3V
  - GND → GND
- **LED and Buzzer:**
  - LED Anode (via 220Ω resistor) → GPIO2
  - LED Cathode → GND
  - Buzzer Positive → GPIO15
  - Buzzer Negative → GND
- **LCD (I2C-based):**
  - SDA → GPIO21
  - SCL → GPIO22
- **Servo Motor:**
  - Signal → GPIO13
  - VCC → 5V (External)
  - GND → Common GND
- **RTC Module (DS1302):**
  - VCC → 3.3V
  - GND → GND
  - CLK → GPIO14
  - DAT → GPIO27
  - RST → GPIO26



FINAL CODE:

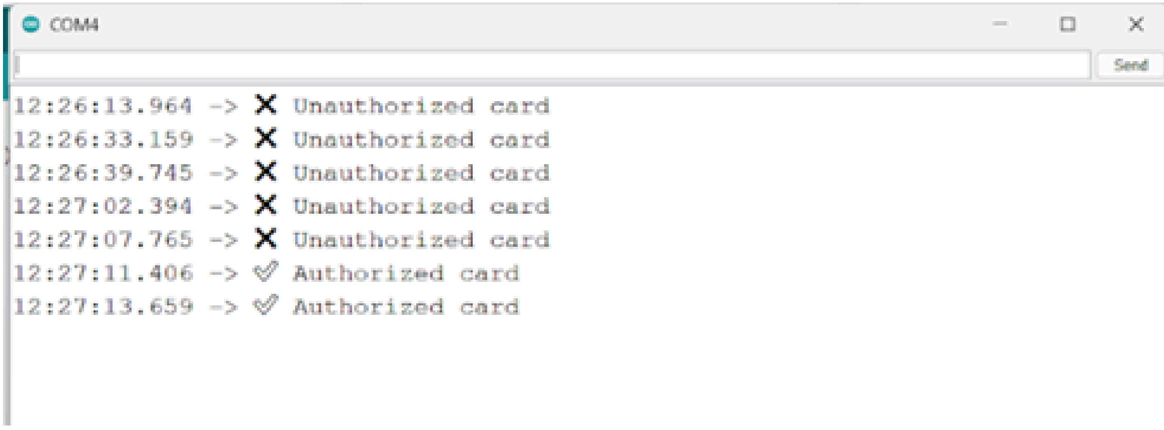
```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <SPI.h>
#include <MFRC522.h>
#include <ESP32Servo.h>
#include <ThreeWire.h>
#include <RtcDS1302.h>
#include <WiFi.h>
#include <HTTPClient.h>

// --- Pin Definitions ---
#define SS_PIN 5        // RFID SS (SDA)
#define RST_PIN 4       // RFID RST
#define LED_PIN 2       // LED Indicator
#define BUZZER_PIN 15   // Buzzer
#define SERVO_PIN 13    // Servo Motor
#define DS1302_CLK 14   // RTC Clock
#define DS1302_IO 27    // RTC Data
#define DS1302_CE 26    // RTC Chip Enable

// --- Wi-Fi Credentials ---
const char* ssid = "Vivo Y22";
const char* password = "ilamathi";
const String scriptURL =
"https://docs.google.com/spreadsheets/d/1A2B3C4D5E6F7G8H9I0J1K2L3M4N/edit";

// --- Known RFID UID ---
byte knownUID[] = {0xED, 0x0D, 0x0C, 0x32};

// --- Components ---
LiquidCrystal_I2C lcd(0x27, 16, 2);       // LCD 16x2 at I2C address 0x27
MFRC522 mfrc522(SS_PIN, RST_PIN);         // RFID module
Servo gateServo;
ThreeWire myWire(DS1302_IO, DS1302_CLK, DS1302_CE);
RtcDS1302<ThreeWire> Rtc(myWire);

// --- Setup ---
void setup() {
  Serial.begin(115200);
  SPI.begin();
  mfrc522.PCD_Init();

  // Setup pins
  pinMode(LED_PIN, OUTPUT);
  pinMode(BUZZER_PIN, OUTPUT);

  // Setup LCD
```

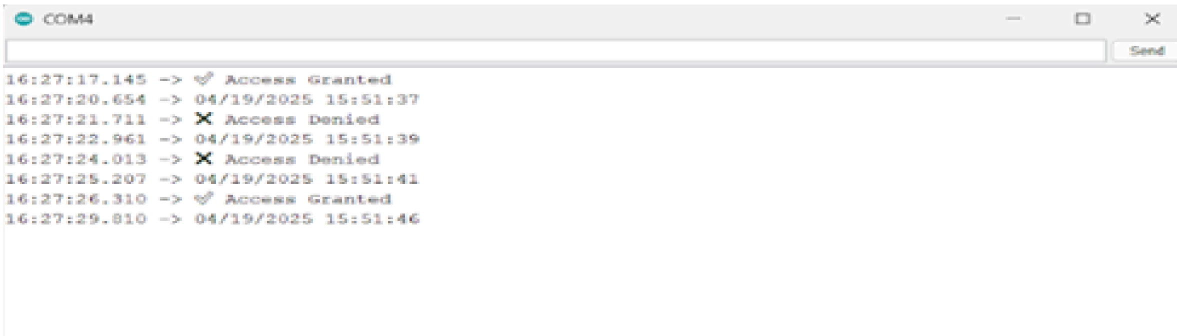
```
  lcd.begin();
  lcd.backlight();
  lcd.setCursor(0, 0);
  lcd.print("Initializing...");

  // Setup Servo
  gateServo.setPeriodHertz(50);
  gateServo.attach(SERVO_PIN, 500, 2400);
  gateServo.write(0);

  // Setup RTC
  Rtc.Begin();
  if (!Rtc.GetIsRunning()) {
    Rtc.SetIsRunning(true);
  }

  // Connect to Wi-Fi
  connectToWiFi();
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Scan your card");
}

// --- Connect to Wi-Fi ---
void connectToWiFi() {
  WiFi.begin(ssid, password);
  Serial.print("Connecting to WiFi");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nWiFi connected.");
}

// --- Main Loop ---
void loop() {
  if (!mfrc522.PICC_IsNewCardPresent() || !mfrc522.PICC_ReadCardSerial()) {
    return;
  }

  String uidStr = getUIDString();
  bool isAuthorized = checkUID(mfrc522.uid.uidByte);

  // Display UID
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("UID: ");
  lcd.print(uidStr);
```

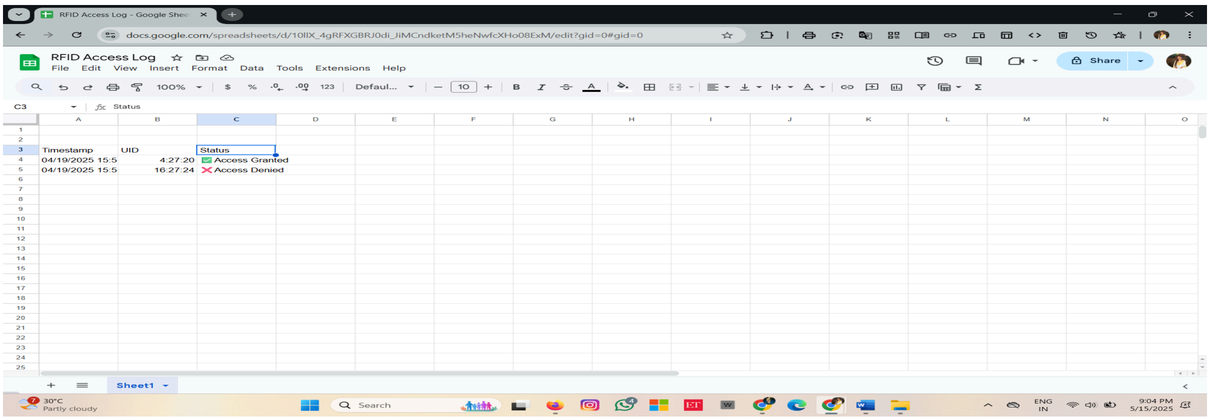
```
  // Handle Access
  if (isAuthorized) {
    handleAccess("✅authorized card", uidStr);
  } else {
    handleAccess("❌Unauthorised card", uidStr);
  }

  mfrc522.PICC_HaltA();
  mfrc522.PCD_StopCrypto1();
  delay(1000);
}

// --- Get UID as String ---
String getUIDString() {
  String uidStr = "";
  for (byte i = 0; i < mfrc522.uid.size; i++) {
    if (mfrc522.uid.uidByte[i] < 0x10) uidStr += "0";
    uidStr += String(mfrc522.uid.uidByte[i], HEX);
    if (i < mfrc522.uid.size - 1) uidStr += ":";
  }
  return uidStr;
}

// --- Check UID ---
bool checkUID(byte* uid) {
  for (byte i = 0; i < 4; i++) {
    if (uid[i] != knownUID[i]) return false;
  }
  return true;
}

// --- Handle Access ---
void handleAccess(String status, String uid) {
  lcd.setCursor(0, 1);
  lcd.print("Access: " + status);

  if (status == "Granted") {
    Serial.println("✅Access granted");
    digitalWrite(LED_PIN, HIGH);
    digitalWrite(BUZZER_PIN, HIGH);
    gateServo.write(180);   // Open gate
    delay(500);
    digitalWrite(LED_PIN, LOW);
    digitalWrite(BUZZER_PIN, LOW);
    delay(3000);
    gateServo.write(0);     // Close gate
  } else {
```

```
    Serial.println("❌Access denied");
    for (int i = 0; i < 3; i++) {
      digitalWrite(LED_PIN, HIGH);
      digitalWrite(BUZZER_PIN, HIGH);
      delay(200);
      digitalWrite(LED_PIN, LOW);
      digitalWrite(BUZZER_PIN, LOW);
      delay(200);
    }
  }
  sendToGoogleSheet(uid, status);
}

// --- Send Data to Google Sheets ---
void sendToGoogleSheet(String uid, String status) {
  if (WiFi.status() == WL_CONNECTED) {
    HTTPClient http;
    String fullURL = scriptURL + "?uid=" + uid + "&status=" + status;
    http.begin(fullURL);
    int httpCode = http.GET();
    if (httpCode > 0) {
      Serial.println("Data sent: " + String(httpCode));
    } else {
      Serial.println("Failed to send. Code: " + String(httpCode));
    }
    http.end();
  } else {
    Serial.println("WiFi not connected");
  }
}
```

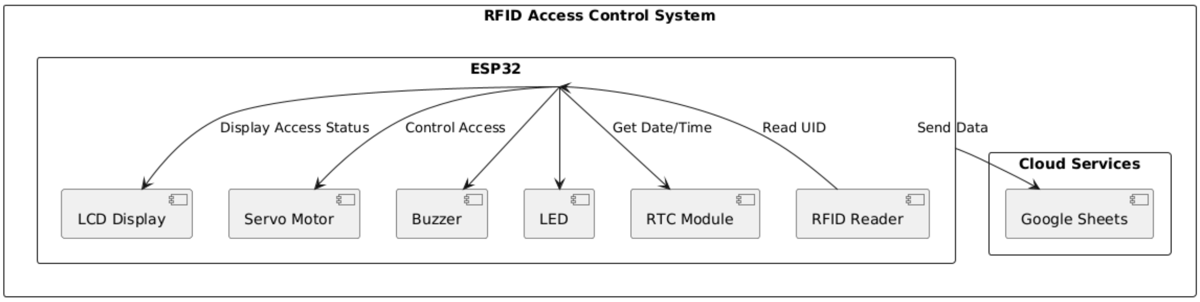
OUTPUT:


COM4
Send
12:26:13.964 -> ✗ Unauthorized card
12:26:33.159 -> ✗ Unauthorized card
12:26:39.745 -> ✗ Unauthorized card
12:27:02.394 -> ✗ Unauthorized card
12:27:07.765 -> ✗ Unauthorized card
12:27:11.406 -> ✅ Authorized card
12:27:13.659 -> ✅ Authorized card


COM4
Send
16:27:17.145 -> ✅ Access Granted
16:27:20.654 -> 04/19/2025 15:51:37
16:27:21.711 -> ✗ Access Denied
16:27:22.961 -> 04/19/2025 15:51:39
16:27:24.013 -> ✗ Access Denied
16:27:25.207 -> 04/19/2025 15:51:41
16:27:26.310 -> ✅ Access Granted
16:27:29.810 -> 04/19/2025 15:51:46


RFID Access Log
File Edit View Insert Format Data Tools Extensions Help
Share
Timestamp UID Status
04/19/2025 15:5 4:27:20 Access Granted
04/19/2025 15:5 16:27:24 Access Denied
Sheet1

BLOCK DIAGRAM:



FLOWCHART: